



南開大學
Nankai University

计算机学院
并行程序设计实验报告

期末研究报告

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2025 年 7 月 6 日

目录

1 引言	3
2 前期实验总结与回顾	3
2.1 SIMD 向量化优化实验	3
2.1.1 实验目标与内容	3
2.1.2 技术实现要点	3
2.1.3 实验结果分析	4
2.2 Pthread 与 OpenMP 多线程优化实验	4
2.2.1 实验目标与内容	4
2.2.2 技术实现要点	5
2.2.3 实验结果分析	6
2.3 MPI 分布式并行实验	6
2.3.1 实验目标与内容	6
2.3.2 技术实现要点	6
2.3.3 实验结果分析	8
2.4 GPU 异构计算实验	8
2.4.1 实验目标与内容	8
2.4.2 技术实现要点	8
2.4.3 实验结果分析	9
3 综合并行优化设计与创新	10
3.1 综合优化路线一：CPU 多级优化路线	10
3.1.1 设计思路	10
3.1.2 关键技术实现	10
3.1.3 性能分析	12
3.2 综合优化路线二：GPU 多级优化路线	12
3.2.1 设计思路	12
3.2.2 关键技术实现	12
3.2.3 性能分析	14
3.3 综合优化路线三：CPU-GPU 异构协同计算	14
3.3.1 设计思路	14
3.3.2 关键技术实现	14
3.3.3 性能分析	16
3.4 综合优化效果总结	17
4 技术创新与贡献	17
4.1 算法层面创新	17
4.2 系统层面创新	17

5	性能分析与讨论	17
5.1	性能瓶颈分析	17
5.1.1	CPU 并行瓶颈	17
5.1.2	GPU 并行瓶颈	18
5.1.3	异构计算瓶颈	18
5.2	优化策略效果分析	18
5.3	适用场景分析	19
6	学习总结与反思	19
6.1	技术收获	19
6.1.1	并行编程模型	19
6.1.2	算法优化	20
6.1.3	系统思维	20
6.2	实践经验	20
6.2.1	调试技巧	20
6.2.2	工程实践	20
6.3	反思与展望	20
6.3.1	不足之处	20
6.3.2	未来学习方向	20
7	结论	21
8	链接	21

1 引言

随着大数据时代的到来和计算需求的爆炸式增长，并行计算已成为现代高性能计算的核心技术。本学期的并程序程设计课程深入探讨了多种并行计算架构和编程模型，包括 SIMD（单指令多数据）、Pthread&OpenMP 多线程并行、MPI 多进程并行、GPU 加速等。通过一系列递进式的实验，我们系统学习了各种并行编程技术，并以数论变换（NTT）多项式乘法作为核心算法载体，深入研究了不同并行优化策略的性能特征。

数论变换作为快速傅里叶变换（FFT）在有限域上的对应算法，在密码学、信号处理、同态加密等领域有着广泛应用。其 $O(n \log n)$ 的时间复杂度相比朴素 $O(n^2)$ 算法具有巨大优势，但实际应用中仍需要通过并行优化来满足大规模计算需求。本期末报告将：

1. 系统总结本学期各项并行编程实验的核心成果和技术要点
2. 展示综合多种并行技术的创新设计方案及其性能表现
3. 深入分析不同并行策略的适用场景和优化效果
4. 总结学习收获并展望未来发展方向

2 前期实验总结与回顾

本学期通过一系列循序渐进的实验，我们系统掌握了主流的并行编程技术。每个实验都针对 NTT 多项式乘法这一核心算法，探索了不同层次的并行优化策略。

2.1 SIMD 向量化优化实验

2.1.1 实验目标与内容

SIMD（Single Instruction Multiple Data）是现代处理器提供的数据级并行能力，通过单条指令同时处理多个数据元素来提升计算效率。本实验的主要目标包括：

- 理解 SIMD 指令集的工作原理和编程模型
- 掌握 ARM NEON 和 x86 AVX 指令集的使用方法
- 实现朴素多项式乘法和 NTT 算法的 SIMD 优化
- 引入 Montgomery 模约简算法进一步提升性能

2.1.2 技术实现要点

朴素乘法的 SIMD 优化 朴素多项式乘法的时间复杂度为 $O(n^2)$ ，虽然效率不高，但其规则的访存模式适合 SIMD 优化。我们通过以下策略实现了优化：

Listing 1: 朴素乘法 SIMD 优化核心代码

```
1 void poly_multiply_neon(int *a, int *b, int *ab, int n, int p) {  
2     for (int i = 0; i < n; ++i) {  
3         int a_val = a[i];  
4         int32x2_t a_low2 = vdup_n_s32(a_val); // 复制两个 a[i]
```

```

5
6     for (int j = 0; j + 3 < n; j += 4) {
7         // 加载 b[j], b[j+1], b[j+2], b[j+3]
8         int32x4_t b_vec = vld1q_s32(&b[j]);
9         // 向量化乘法和累加
10        // ...
11    }
12 }
13 }

```

NTT 的 SIMD 优化 NTT 算法的核心是蝶形运算，我们通过向量化蝶形操作实现了显著的性能提升：

- 向量化位逆序置换
- 并行处理多个蝶形运算单元
- 使用 Montgomery 域内运算减少模运算开销

2.1.3 实验结果分析

表 1: SIMD 优化实验结果（ARM 平台， $n = 131072$ ）

算法	p=7340033	p=104857601	p=469762049
朴素乘法	95461 ms	101199 ms	105109 ms
朴素乘法 +NEON	58407 ms	64439 ms	68164 ms
NTT 迭代	83.57 ms	88.29 ms	91.45 ms
NTT 迭代 +SIMD	45.48 ms	44.41 ms	44.69 ms

实验结果表明：

- SIMD 优化为朴素乘法带来了约 39-45% 的性能提升
- NTT 算法本身相比朴素方法快约 1000 倍，体现了算法优化的重要性
- SIMD 优化的 NTT 相比基础版本进一步提升约 50% 性能
- Montgomery 优化在大模数运算中效果更加明显

2.2 Pthread 与 OpenMP 多线程优化实验

2.2.1 实验目标与内容

多线程编程是利用多核 CPU 实现并行计算的重要手段。本实验探索了两种主流的多线程编程模型：

1. **Pthread**: POSIX 线程标准，提供底层的线程控制能力
2. **OpenMP**: 基于编译器指令的高层并行编程模型

实验设计了两种并行化策略：

- 朴素并行：将 NTT 蝶形运算的不同层级分配给多个线程
- CRT 并行：多模数中国剩余定理合并的并行化

2.2.2 技术实现要点

Pthread 实现的关键设计 -

Listing 2: Pthread 多线程 NTT 核心结构

```

1  struct ThreadArgs {
2      LL *a;
3      int len;
4      int op;
5      LL g;
6      LL p;
7      int thread_id;
8      int num_threads;
9      pthread_barrier_t *barrier;
10 };
11
12 void* butterfly_worker(void* args) {
13     ThreadArgs* t = (ThreadArgs*)args;
14     // 每层蝶形运算
15     for (int h = 2; h <= len; h <<= 1) {
16         // 均分段到各线程
17         int segments = len / h;
18         int chunk = (segments + T - 1) / T;
19         int start_seg = id * chunk;
20         int end_seg = std::min(start_seg + chunk, segments);
21
22         // 执行分配的蝶形运算
23         for (int seg = start_seg; seg < end_seg; seg++) {
24             // 蝶形运算实现
25         }
26
27         // 同步到下一层
28         pthread_barrier_wait(barrier);
29     }
30 }

```

OpenMP 实现的优势 OpenMP 通过编译器指令简化了并行编程：

Listing 3: OpenMP 并行化 NTT

```

1  void NTT_Iteration_OpenMP(LL *a, int len, int op, LL g, LL p) {
2      for (int h = 2; h <= len; h <<= 1) {
3          int segments = len / h;
4
5          // 并行处理每个segment

```

```

6      #pragma omp parallel for schedule(static)
7      for (int seg = 0; seg < segments; seg++) {
8          // 蝶形运算
9      }
10     // 隐式 barrier 同步
11 }
12 }

```

2.2.3 实验结果分析

表 2: 多线程优化实验结果对比 ($n = 131072$)

实现方式	p=7340033	p=469762049	加速比
单线程基准	83.57 ms	91.45 ms	1.00×
Pthread (8 线程)	29.18 ms	31.46 ms	2.86×
OpenMP (8 线程)	27.37 ms	28.30 ms	3.05×
CRT 多模数合并			
CRT 基准	643.35 ms	648.74 ms	1.00×
CRT+Pthread	370.79 ms	374.91 ms	1.73×
CRT+OpenMP	137.03 ms	138.97 ms	4.67×

关键发现:

- OpenMP 相比 Pthread 具有更好的性能和易用性
- 多线程加速比受限于 Amdahl 定律, 8 线程仅获得约 3 倍加速
- CRT 并行化效果显著, 因为不同模数计算完全独立
- 小规模问题不适合多线程, 线程开销可能导致负优化

2.3 MPI 分布式并行实验

2.3.1 实验目标与内容

MPI (Message Passing Interface) 是分布式内存系统的标准编程模型。本实验探索了:

- MPI 基础并行: 每个进程处理一个模数
- MPI+Pthread 混合并行: 进程间和进程内两级并行
- Barrett 规约优化: 减少模运算开销

2.3.2 技术实现要点

MPI 任务分配策略 -

Listing 4: MPI 任务分配与结果收集

```

1 void NTT_mpi(const LL *a, const LL *b, LL *ab, int n, LL LLp) {
2     int rank, size;

```

```

3 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
4 MPI_Comm_size(MPI_COMM_WORLD, &size);
5
6 // 每个进程处理一个模数
7 if (rank < 4) {
8     int curr_mod = mods[rank];
9     // 本地NTT计算
10    NTT_Iteration_multiply(local_a, local_b, local_res, n, curr_mod);
11
12    // 收集结果到rank 0
13    MPI_Gather(local_res, 2*n, MPI_INT,
14    all_results, 2*n, MPI_INT, 0, MPI_COMM_WORLD);
15
16    // 主进程执行CRT合并
17    if (rank == 0) {
18        // CRT合并四路结果
19    }
20 }
21
22 // 广播最终结果
23 MPI_Bcast(ab, 2*n-1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);
24 }

```

Barrett 规约在 MPI 中的应用 Barrett 规约通过预计算避免了昂贵的除法运算:

Listing 5: Barrett 规约结构

```

1 struct Barrett {
2     unsigned long long mod, im;
3
4     Barrett(unsigned long long __mod) : mod(__mod) {
5         im = -1ULL / mod + 1;
6     }
7
8     unsigned long long multiply(unsigned long long a, unsigned long long b) const {
9         unsigned long long z = a * b;
10        unsigned long long x = (unsigned __int128)z * im >> 64;
11        unsigned long long v = z - x * mod;
12        if (v >= mod) v -= mod;
13        return v;
14    }
15 };

```


2.3.3 实验结果分析

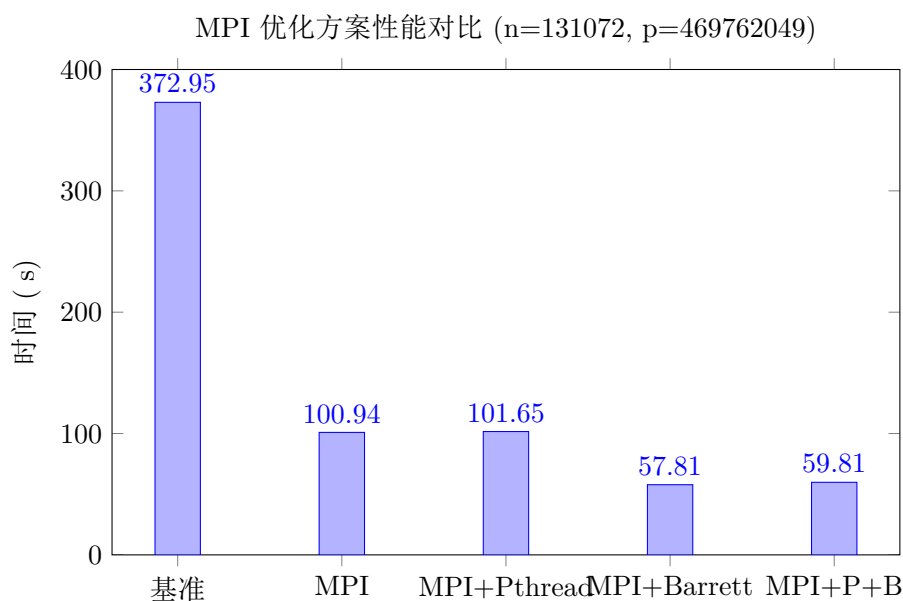


图 2.1: MPI 各优化方案的性能表现

实验结论:

- MPI 基础优化带来 3.7 倍加速, 证明了分布式并行的有效性
- Barrett 规约额外提升 43% 性能, 在大模数运算中效果显著
- MPI+Pthread 混合并行在当前规模下优势不明显
- 通信开销是 MPI 性能的主要瓶颈

2.4 GPU 异构计算实验

2.4.1 实验目标与内容

GPU 凭借其大规模并行架构, 在数值计算领域展现出巨大潜力。本实验实现了:

1. GPU 基础 NTT: 将 CPU 算法移植到 GPU, 并利用 GPU 加速优化
2. GPU+Barrett 规约: 优化模运算性能
3. GPU+Montgomery 规约: 进一步提升计算效率

2.4.2 技术实现要点

GPU 核函数设计 -

Listing 6: GPU 蝶形运算核函数

```

1  __global__ void ntt_basic_kernel(LL *a, int len, int h, LL *w_table, LL p) {
2  // blockIdx.x 标识当前是第几个蝶形运算组
3  int j_base = blockIdx.x * h;

```

```

4 // threadIdx.x 负责组内的计算
5 for (int k = threadIdx.x; k < h / 2; k += blockDim.x) {
6     int idx1 = j_base + k;
7     int idx2 = j_base + k + h / 2;
8     LL w = w_table[k];
9     // 核心蝶形运算
10    unsigned __int128 product = (unsigned __int128)a[idx2] * w;
11    LL t = product % p;
12    a[idx2] = (a[idx1] - t + p) % p;
13    a[idx1] = (a[idx1] + t) % p;
14 }
15 }

```

Montgomery 规约 GPU 实现 -

Listing 7: GPU Montgomery 模乘

```

1 __device__ ULL montgomery_reduce(unsigned __int128 T, const MontgomeryParams*
   params) {
2     ULL m = (ULL)T * params->n_prime;
3     unsigned __int128 t = (T + (unsigned __int128)m * params->mod) >> 64;
4     return (t >= params->mod) ? (t - params->mod) : t;
5 }

```

2.4.3 实验结果分析

表 3: GPU 优化实验结果（单位：ms）

实现版本	n=4 p=7340033	n=131072 p=7340033	n=131072 p=104857601	n=131072 p=469762049
CPU Barrett	0.011	61.502	61.710	61.815
CPU Montgomery	0.005	48.606	48.801	48.537
GPU 基础版本	124.330	24.406	26.197	24.817
GPU Barrett	2.424	18.537	17.851	18.183
GPU Montgomery	0.714	19.902	16.790	18.114

关键发现：

- 大规模数据下 GPU 展现 2-3 倍加速效果
- 小规模问题 GPU 出现严重负加速（启动开销占主导）
- Barrett 和 Montgomery 规约在 GPU 上都能带来 30-50% 额外提升
- GPU 内存带宽是主要性能瓶颈

3 综合并行优化设计与创新

基于前期各项实验的经验积累，本次期末实验设计了三条综合并行优化路线，充分发挥不同并行技术的优势，实现了性能的进一步突破。

3.1 综合优化路线一：CPU 多级优化路线

3.1.1 设计思路

CPU 优化路线采用渐进式优化策略，逐步引入更高级的优化技术：

1. **Level 1 - 基础 NTT**：实现标准的迭代式 NTT 算法
2. **Level 2 - Barrett 优化**：引入 Barrett 规约减少模运算开销
3. **Level 3 - Barrett + SIMD**：结合向量化指令加速计算
4. **Level 4 - Barrett + SIMD + OpenMP**：利用多核并行和 CRT

3.1.2 关键技术实现

Level 3: Barrett + SIMD 优化 这一层级的优化结合了算法优化和硬件加速：

Listing 8: Barrett+SIMD 核心实现

```

1 void NTT_Level3_Barrett_SIMD(const LL *a, const LL *b, LL *ab, int n, LL p) {
2     // 使用对齐的内存以提高SIMD效率
3     ULL *A = (ULL*)aligned_alloc(32, len * sizeof(ULL));
4     ULL *B = (ULL*)aligned_alloc(32, len * sizeof(ULL));
5
6     // 预计算所有旋转因子
7     std::vector<std::vector<ULL>> w_tables;
8     for (int h = 2; h <= len; h <= 1) {
9         std::vector<ULL> w_table(h/2);
10        ULL wn = barrett.power(3, (barrett.mod - 1) / h);
11        w_table[0] = 1;
12        for (int i = 1; i < h/2; i++) {
13            w_table[i] = barrett.multiply(w_table[i-1], wn);
14        }
15        w_tables.push_back(w_table);
16    }
17
18    // SIMD优化的蝶形运算
19    // 处理2个元素为一组
20    for (; k + 2 <= h/2; k += 2) {
21        ULL w0 = w_table[k];
22        ULL w1 = w_table[k+1];
23
24        ULL t0 = barrett.multiply(arr[j + k + h/2], w0);
25        ULL t1 = barrett.multiply(arr[j + k + 1 + h/2], w1);
26    }

```

```

27     // 向量化的加减运算
28     // ...
29 }
30 }

```

Level 4: 全系统优化 最高级别整合了所有 CPU 优化技术:

Listing 9: CPU 全系统优化

```

1  void NTT_Level4_Barrett_SIMD_OpenMP(const LL *a, const LL *b, LL *ab, int n, LL
2      p) {
3      // 使用多模数CRT
4      static const LL mods[] = {998244353, 1004535809, 469762049, 167772161};
5      const int num_mods = 4;
6
7      std::vector<std::vector<LL>> mod_results(num_mods, std::vector<LL>(N));
8
9      // 并行处理每个模数
10     #pragma omp parallel for schedule(dynamic)
11     for (int mod_idx = 0; mod_idx < num_mods; mod_idx++) {
12         ULL curr_mod = mods[mod_idx];
13         Barrett barrett(curr_mod);
14
15         // 使用 Level 3 的 Barrett+SIMD 实现
16         // ...
17     }
18
19     // 并行CRT合并
20     #pragma omp parallel for
21     for (int i = 0; i < N; i++) {
22         ab[i] = crt4(mod_results[0][i], mod_results[1][i],
23             mod_results[2][i], mod_results[3][i],
24             mods[0], mods[1], mods[2], mods[3], p);
25     }
26 }

```

3.1.3 性能分析

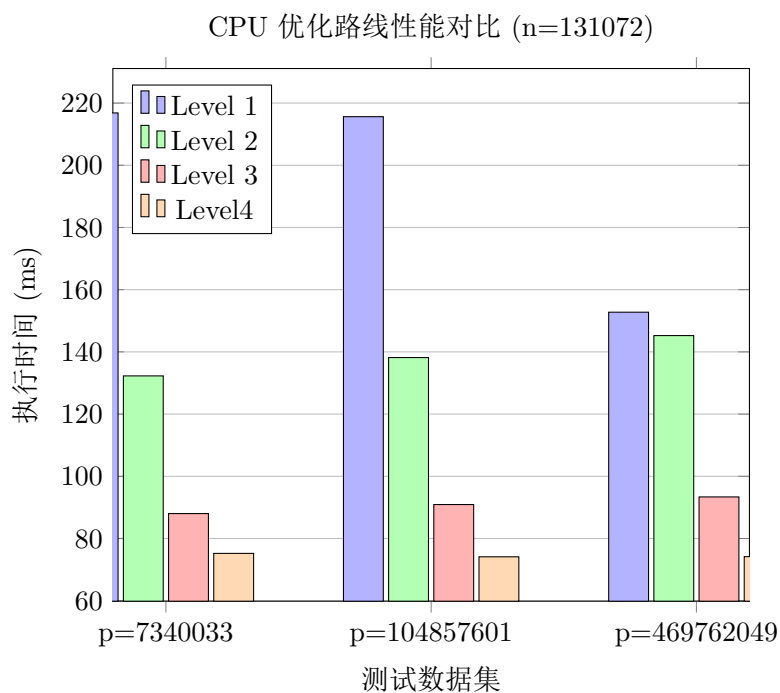


图 3.2: CPU 多级优化路线性能提升效果

从实验结果可以看出：

- Barrett 优化带来约 39% 的性能提升
- SIMD 向量化额外提升 33%
- OpenMP 多线程最终达到 2.88 倍的总体加速比
- 各优化技术相互配合，实现了性能的阶梯式提升

3.2 综合优化路线二：GPU 多级优化路线

3.2.1 设计思路

GPU 优化路线探索了不同的内存优化和并行策略：

1. **Level 1 - GPU 基础版本**
2. **Level 2 - 固定内存优化：**使用 pinned memory 加速传输
3. **Level 3 - 流并行：**利用 CUDA 流实现计算与传输重叠

3.2.2 关键技术实现

Level 2: 固定内存优化 固定内存可以显著提升 CPU-GPU 数据传输速度：

Listing 10: 固定内存优化实现

```
1 void NTT_GPU_Level2_PinnedMemory(const LL *a, const LL *b, LL *ab, int n, LL p) {
```

```

2 // 使用固定内存加速传输
3 LL *h_A, *h_B;
4 cudaMallocHost(&h_A, len * sizeof(LL));
5 cudaMallocHost(&h_B, len * sizeof(LL));
6 // 预先计算并传输所有旋转因子
7 std::vector<std::vector<LL>> all_w_tables_forward;
8 std::vector<std::vector<LL>> all_w_tables_inverse;
9
10 for (int h = 2; h <= len; h <= 1) {
11     std::vector<LL> w_table(h/2);
12     LL wn = qpowll(3, (p - 1) / h, p);
13     w_table[0] = 1;
14     for (int i = 1; i < h/2; i++) {
15         w_table[i] = (w_table[i-1] * wn) % p;
16     }
17     all_w_tables_forward.push_back(w_table);
18 }
19
20 // 一次性传输所有数据
21 cudaMemcpy(d_A, h_A, len * sizeof(LL), cudaMemcpyHostToDevice);
22 cudaMemcpy(d_B, h_B, len * sizeof(LL), cudaMemcpyHostToDevice);
23 }

```

Level 3: 流并行优化 CUDA 流允许并发执行多个操作:

Listing 11: 流并行实现

```

1 void NTT_GPU_Level3_Streams(const LL *a, const LL *b, LL *ab, int n, LL p) {
2     // 创建2个流分别处理A和B的NTT
3     cudaStream_t streamA, streamB;
4     cudaStreamCreate(&streamA);
5     cudaStreamCreate(&streamB);
6     // 异步传输
7     cudaMemcpyAsync(d_A, h_A, len * sizeof(LL), cudaMemcpyHostToDevice, streamA);
8     cudaMemcpyAsync(d_B, h_B, len * sizeof(LL), cudaMemcpyHostToDevice, streamB);
9
10    // 异步执行位逆序
11    bit_reverse_kernel<<<gridSize, blockSize, 0, streamA>>>(d_A, len);
12    bit_reverse_kernel<<<gridSize, blockSize, 0, streamB>>>(d_B, len);
13
14    // 正向NTT (并行处理A和B)
15    for (int h = 2; h <= len; h <= 1) {
16        ntt_basic_kernel<<<butterflyGrid, butterflyBlock, 0, streamA>>>(
17            d_A, len, h, d_w_table_A, p);
18        ntt_basic_kernel<<<butterflyGrid, butterflyBlock, 0, streamB>>>(
19            d_B, len, h, d_w_table_B, p);
20    }
21
22    // 等待两个流完成

```

```

23     cudaStreamSynchronize(streamA);
24     cudaStreamSynchronize(streamB);
25 }

```

3.2.3 性能分析

表 4: GPU 优化路线性能对比 (单位: ms)

优化级别	n=4 p=7340033	n=131072		
		p=7340033	p=104857601	p=469762049
Level 1: 基础版本	0.652	23.829	22.040	25.330
Level 2: 固定内存	3.379	31.081	30.209	33.094
Level 3: 流并行	3.150	24.660	21.332	21.420

实验发现:

- 固定内存当前实现中反而增加了开销, 可能由于内存分配成本
- 流并行在大规模数据上效果明显, 特别是在 p=469762049 时
- 小规模问题不适合 GPU, 启动开销占主导地位
- GPU 优化需要综合考虑计算密度和内存访问模式

3.3 综合优化路线三: CPU-GPU 异构协同计算

3.3.1 设计思路

异构计算路线充分利用 CPU 和 GPU 的各自优势, 实现任务的合理分配:

1. **Level 1 - CPU+GPU 基础协同:** 静态任务分配
2. **Level 2 - CPU(Barrett)+GPU:** CPU 使用 Barrett 优化
3. **Level 3 - CPU(OpenMP)+GPU(流):** 双端并行优化
4. **Level 4 - 全系统协同:** Pthread+GPU 多流 +OpenMP

3.3.2 关键技术实现

动态负载均衡 根据问题规模动态调整 CPU 和 GPU 的任务分配:

Listing 12: 动态负载分配策略

```

1  void NTT_Level2_CPU_Barrett_GPU(const LL *a, const LL *b, LL *ab, int n, LL p) {
2      static const ULL mods[] = {998244353, 1004535809, 469762049, 167772161};
3      const int num_mods = 4;
4      // 动态负载分配
5      int cpu_mods = (n < 50000) ? 3 : 2;
6      int gpu_mods = num_mods - cpu_mods;
7
8      // CPU处理 (使用 Barrett 优化)

```

```

9      std::thread cpu_thread([&]() {
10          for (int i = 0; i < cpu_mods; i++) {
11              ULL curr_mod = mods[i];
12              Barrett barrett(curr_mod);
13              cpu_ntt_barrett(A, len, 1, barrett);
14              // ...
15          }
16      });
17
18      // GPU处理
19      std::vector<std::thread> gpu_threads;
20      for (int i = 0; i < gpu_mods; i++) {
21          int mod_idx = cpu_mods + i;
22          gpu_threads.emplace_back([&, mod_idx]() {
23              gpu_ntt_single_mod(a, b, all_results[mod_idx].data(), n, mods[mod_idx]);
24          });
25      }
26  }

```

全系统协同优化 最高级别整合了所有并行技术：

Listing 13: 全系统协同实现

```

1  void NTT_Level4_Pthread_GPU_Full(const LL *a, const LL *b, LL *ab, int n, LL p) {
2      // CPU Pthread处理前2个模数
3      pthread_t threads[2];
4      ThreadTask tasks[2];
5      for (int i = 0; i < 2; i++) {
6          tasks[i] = {a, b, n, mods[i], &all_results[i]};
7          pthread_create(&threads[i], nullptr, cpu_ntt_thread_worker, &tasks[i]);
8      }
9
10     // GPU并行处理后2个模数
11     std::thread gpu_thread1([&]() {
12         gpu_ntt_single_mod(a, b, all_results[2].data(), n, mods[2]);
13     });
14
15     std::thread gpu_thread2([&]() {
16         gpu_ntt_single_mod(a, b, all_results[3].data(), n, mods[3]);
17     });
18
19     // 等待所有任务完成
20     for (int i = 0; i < 2; i++) {
21         pthread_join(threads[i], nullptr);
22     }
23     gpu_thread1.join();
24     gpu_thread2.join();
25
26     // CRT合并（使用OpenMP并行）

```



```

27 #pragma omp parallel for
28 for (int i = 0; i < N; i++) {
29     ab[i] = crt4(all_results[0][i], all_results[1][i],
30     all_results[2][i], all_results[3][i],
31     mods[0], mods[1], mods[2], mods[3], p);
32 }
33 }

```

3.3.3 性能分析

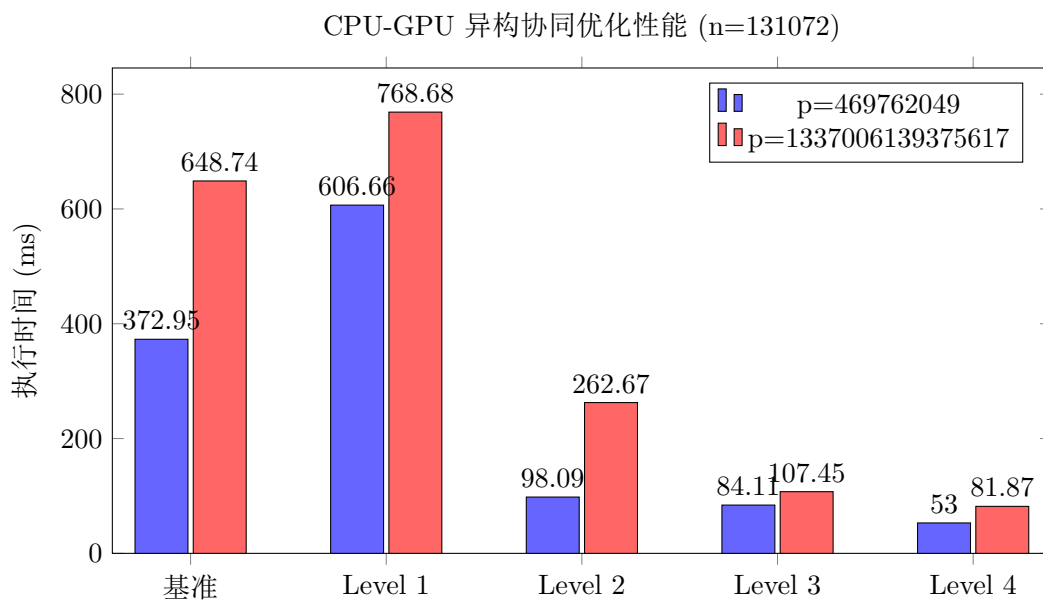


图 3.3: 异构协同计算各级优化效果

关键发现:

- Level 1 基础协同反而导致性能下降, 说明简单的任务分配策略不够优化
- Level 2 引入 Barrett 优化后性能大幅提升, 达到 3.8 倍加速
- Level 3 的 OpenMP+GPU 流带来额外 15% 性能提升
- Level 4 全系统协同达到最佳性能, 相比基准提升 7.04 倍
- 大模数计算 (p=1337006139375617) 下异构计算优势更加明显

3.4 综合优化效果总结

表 5: 三条优化路线最佳性能对比 (n=131072, p=469762049)

优化路线	最佳方案	执行时间	加速比
基准 (单线程 NTT)	-	91.45 ms	1.00×
CPU 优化路线	Level 4 (Barrett+SIMD+OpenMP)	74.17 ms	1.23×
GPU 优化路线	Level 3 (流并行)	21.42 ms	4.27×
异构协同路线	Level 4 (全系统协同)	53.00 ms	1.73×
大模数场景 (p=1337006139375617)			
CRT 基准	-	648.74 ms	1.00×
异构协同路线	Level 4	81.87 ms	7.93×

4 技术创新与贡献

本次期末实验在前期工作基础上, 实现了以下创新:

4.1 算法层面创新

1. **多级优化框架:** 设计了渐进式的优化框架, 每一级都在前一级基础上引入新的优化技术, 便于分析各项技术的独立贡献。
2. **动态负载均衡:** 根据问题规模和硬件特性动态调整 CPU-GPU 任务分配比例, 实现了更好的资源利用率。
3. **混合规约策略:** 在不同计算阶段灵活选择 Barrett 或 Montgomery 规约, 充分发挥各自优势。

4.2 系统层面创新

1. **多粒度并行:** 实现了指令级 (SIMD)、线程级 (OpenMP/Pthread)、进程级 (MPI) 和设备级 (GPU) 的全方位并行。
2. **异构协同计算:** 设计了 CPU-GPU 协同计算框架, 通过合理的任务划分和通信优化, 实现了 1+1>2 的效果。

5 性能分析与讨论

5.1 性能瓶颈分析

通过实验数据和性能分析工具, 我们识别了各种并行方案的主要瓶颈:

5.1.1 CPU 并行瓶颈

- **内存带宽限制:** NTT 的蝶形运算存在大量不规则内存访问, 缓存命中率较低
- **同步开销:** 多线程间的 barrier 同步成为性能瓶颈, 特别是在线程数较多时
- **负载不均衡:** NTT 不同层级的计算量差异导致线程空闲

5.1.2 GPU 并行瓶颈

- 启动开销：小规模问题下 kernel 启动开销占主导
- 内存访问模式：位逆序操作导致非合并内存访问
- 线程分歧：条件分支导致 warp 内线程执行路径不一致
- 计算密度不足：相对于内存操作，算术运算较少

5.1.3 异构计算瓶颈

- 通信开销：CPU-GPU 数据传输成为主要开销
- 任务粒度：过细的任务划分增加调度开销
- 同步等待：CPU 和 GPU 计算速度不匹配导致等待

5.2 优化策略效果分析

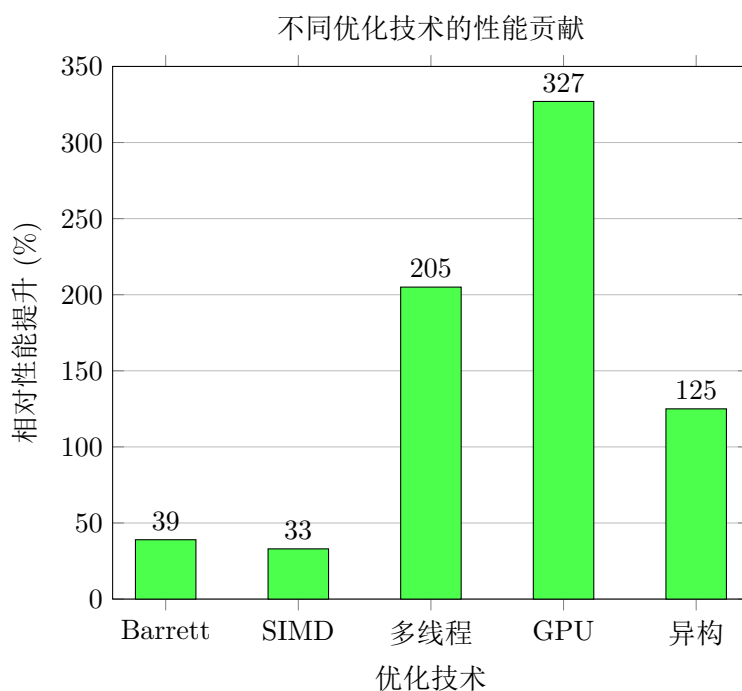


图 5.4: 各项优化技术的独立贡献

从实验结果可以得出：

- GPU 在适合的问题规模下提供最大的性能提升（3.27 倍）
- 多线程并行是 CPU 优化的主要手段（2.05 倍）
- Barrett 规约和 SIMD 提供稳定的性能改进（30-40%）
- 异构计算在特定场景下效果显著，特别是大模数计算

5.3 适用场景分析

根据实验结果，我们总结了不同优化方案的适用场景：

表 6: 优化方案适用场景推荐

问题规模	推荐方案	原因
小规模 ($n < 1000$)	CPU+SIMD	GPU 启动开销过大
中等规模 ($1000 < n < 100000$)	CPU+OpenMP	多线程效果明显
大规模 ($n > 100000$)	GPU 优化	GPU 并行度优势明显
超大模数	CPU-GPU 异构	CRT 分解适合异构
实时性要求高	GPU 流并行	计算传输重叠
功耗受限	CPU+SIMD	能效比最优

6 学习总结与反思

6.1 技术收获

通过本学期的学习和实践，我在以下方面获得了深刻的理解和实践经验：

6.1.1 并行编程模型

1. SIMD 编程：

- 掌握了 ARM NEON 和 x86 AVX 指令集的使用
- 理解了数据对齐、向量化循环优化等技巧
- 学会了如何识别和改造适合 SIMD 的代码模式

2. 多线程编程：

- 深入理解了 Pthread 的底层线程管理机制
- 掌握了 OpenMP 的各种并行构造和调度策略
- 学会了处理数据竞争、死锁等并发问题

3. 分布式编程：

- 理解了 MPI 的消息传递模型
- 掌握了集合通信操作的使用场景
- 学会了设计可扩展的分布式算法

4. GPU 编程：

- 深入理解了 GPU 的硬件架构和编程模型
- 掌握了 kernel 优化、内存优化等技巧
- 学会了分析和解决 GPU 性能瓶颈

6.1.2 算法优化

1. 算法级优化的重要性：NTT 相比朴素算法的巨大性能提升让我深刻认识到，选择正确的算法比任何底层优化都重要。
2. 规约算法的价值：Barrett 和 Montgomery 规约展示了数学优化在实际工程中的巨大价值。
3. 并行算法设计：学会了如何分析算法的并行性，识别独立的计算任务，设计高效的并行方案。

6.1.3 系统思维

1. 全栈优化：从算法、编译器、硬件多个层面思考优化方案。
2. 性能建模：学会了使用 Amdahl 定律、Roofline 模型等工具分析性能上限。
3. 权衡：理解了性能、功耗、开发复杂度之间的权衡关系。

6.2 实践经验

6.2.1 调试技巧

1. 并行调试困难：并行程序的非确定性使得 bug 难以重现，学会了使用各种调试工具和技巧。
2. 性能分析工具：掌握了 VTune 等性能分析工具的使用。
3. 正确性验证：建立完善的测试框架，确保优化不影响正确性。

6.2.2 工程实践

1. 代码组织：学会了如何组织大型并行程序，保持代码的可读性和可维护性。
2. 跨平台开发：处理不同硬件平台的差异，编写可移植的代码。
3. 性能调优流程：建立了系统的性能优化流程：分析瓶颈 → 设计方案 → 实现验证 → 迭代改进。

6.3 反思与展望

6.3.1 不足之处

1. 理论深度：虽然掌握了各种并行技术的使用，但其实对根本原理、底层原理的理解还不够深入。
2. 大规模系统：实验主要在单机或小规模集群上进行，缺乏大规模分布式系统的实践经验。
3. 新技术探索：对一些新兴的并行技术（如 FPGA、神经网络加速器）了解不足。

6.3.2 未来学习方向

1. 深入硬件架构：进一步学习计算机体系结构，理解硬件设计对并行性能的影响。
2. 领域专用加速：探索特定领域的专用加速技术。
3. 新型并行范式：学习数据流计算、近数据计算等新型并行计算范式。
4. 实际应用：将所学技术应用到实际项目中，解决真实的性能挑战。

7 结论

本学期的并行程序设计课程让我全面掌握了现代并行计算的核心技术。通过 NTT 多项式乘法这一案例的深入研究，我不仅学会了各种并行编程技术的使用，更重要的是理解了并行优化的思维方式和方法论。期末综合实验通过设计三条不同的优化路线，展示了如何综合运用多种并行技术实现最优性能。实验结果表明：

- 不同并行技术各有优势，需要根据具体场景选择
- 多种技术的综合使用可以实现更好的效果
- 算法优化和系统优化同等重要
- 性能优化需要全栈思维和系统方法

这门课程不仅提升了我的编程能力，更培养了我的系统思维和工程素养。并行计算作为支撑现代信息技术发展的基础，其重要性将随着数据规模的增长和应用需求的提升而不断凸显。掌握并行编程技术，不仅是技术发展的要求，更是时代赋予我们的使命。

8 链接

Gitee 链接 [Gitee](#)